API Versioning Without Breaking Clients

URL vs header vs date, and how to retire a version

Watch the 4-min video

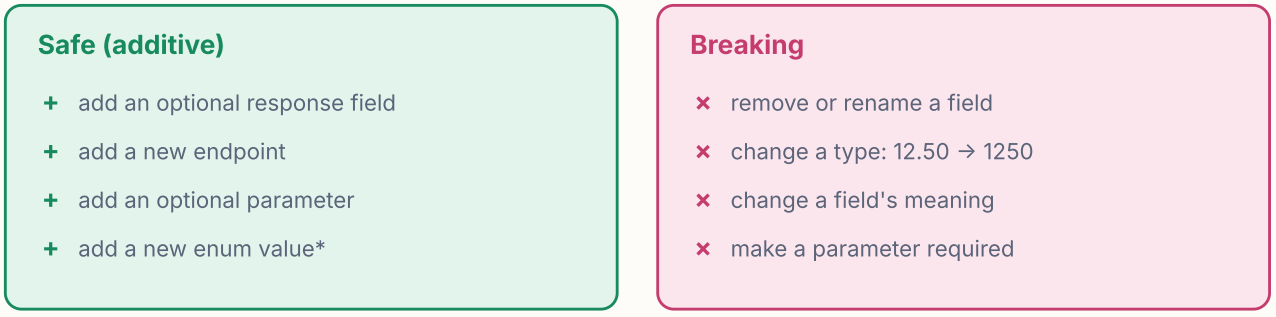
In one line: evolve your API additively so you rarely need a new version; when you must break, run the new version side by side (one shared core, thin adapters) and retire the old one with notice, Deprecation and Sunset headers, usage tracking and brownouts.

1 The problem: a breaking change with live clients

Your payments API sends amounts as a decimal number of dollars: `"amount": 12.50`. You want an integer number of cents plus a currency: `"amount_cents": 1250, "currency": "USD"`. Integers avoid floating-point rounding, and the currency field makes the API multi-currency. But thousands of partner integrations parse the old format, and you cannot make them all change overnight. API versioning lets old and new clients live side by side.

2 Know what actually breaks clients

Before you version anything, ask whether the change is breaking at all. A change is **backward compatible** if every existing client keeps working without touching its code.



** safe only if clients ignore unknown values; strict clients may still break*

Figure 1. Additive changes are safe as long as clients ignore fields they don't know. Anything that removes, renames, retypes or tightens is breaking.

So the best versioning strategy is to **need it rarely**. Add new fields next to old ones, keep the old ones working, and tell clients to be **tolerant readers**: ignore unknown fields instead of failing on them. Many teams ship for years on one version this way.

3 Three ways to signal the version

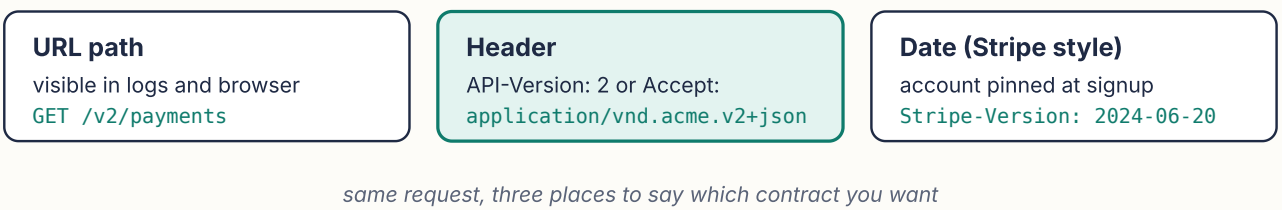


Figure 2. URL path, a header (custom, or a vendor media type in Accept), or a date that pins each account to the API as it was on that day.

- **URL path**: `/v2/payments`. The most common choice for public APIs.
- **Header**: a custom `API-Version: 2`, or content negotiation with `Accept: application/vnd.acme.v2+json`. URLs stay clean, but the version is invisible in a browser and easy to forget in caches.
- **Date**: Stripe pins each account to the API version from the day it signed up. Clients opt in to a newer date when ready, account-wide or per request with a `Stripe-Version` header. This allows many small breaking changes instead of one big v2.

Pros and cons of URL versioning

Pros	Cons
Visible: the version shows in logs, dashboards and bug reports	Purists: a URI should name the resource, not its format
Easy to try in a browser or with curl	Versions the whole API at once, even for one changed endpoint
Easy to route at the gateway by prefix	Every client must change its URLs to upgrade
Caches like any other URL (no `Vary` header needed)	Tends to push you to big, infrequent versions

4 Example: /v2/payments next to /v1 for twelve months

The gateway routes both prefixes. The key design choice: **only one implementation holds the business logic**, written against the v2 model. A thin **adapter** translates v1 requests into v2 and v2 responses back into v1. Bugs get fixed once, not twice, and v1 cannot quietly drift.

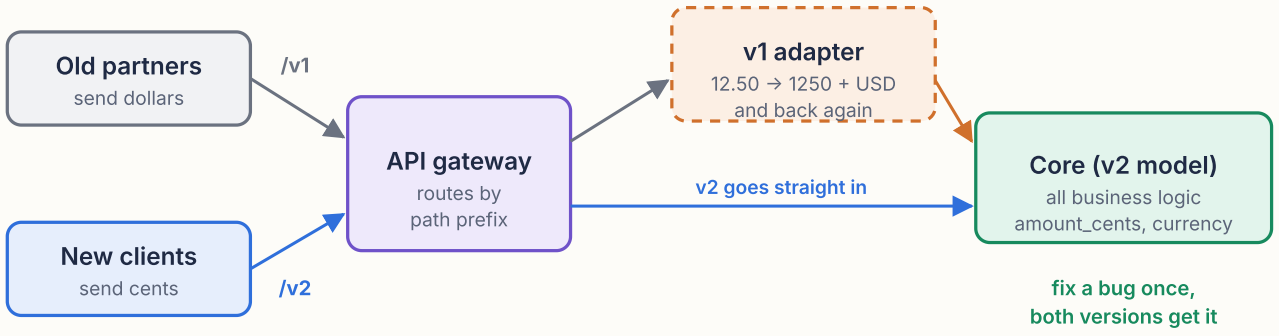


Figure 3. v1 traffic passes through a small adapter that converts dollars to cents on the way in and cents to dollars on the way out; v2 goes straight to the core.

WORKED EXAMPLE

A v1 partner sends `POST /v1/payments {"amount": 12.50}`. The adapter parses the amount as a decimal (not a float), multiplies by 100 and adds the account's default currency: `{"amount_cents": 1250, "currency": "USD"}`. The core charges the card and returns `amount_cents: 1250`; the adapter divides by 100 and the partner sees `"amount": 12.50`, exactly as before.

Cost check: if v1 still carries 30% of 10,000 RPS, the adapter handles 3,000 RPS of pure field mapping, microseconds each. Two full codebases would double every bug fix and security patch.

5 Deprecate politely



Figure 4. A twelve-month retirement. Brownouts are planned windows where v1 returns errors, to flush out forgotten integrations before the final switch-off.

- **Announce** the timeline with a migration guide that maps every v1 field to v2.
- **Mark** every v1 response with a `Deprecation` header and a `Sunset` header carrying the shutdown date, plus a link to the guide.
- **Track** v1 usage per client (API key) and contact the teams still calling it.
- **Brownout** near the end: for example 1 hour, then 4 hours, then a day, where v1 returns errors. Someone always discovers a cron job nobody remembered.
- **Switch off** for good; returning `410 Gone` with a pointer to v2 is clearer than a 404.

```
HTTP/1.1 200 OK
Deprecation: @1782864000
Sunset: Wed, 30 Jun 2027 23:59:59 GMT
Link: <https://docs.acme.com/migrate-v2>; rel="deprecation"
```

6 Rules that keep versioning manageable

- **At most two major versions live** at once. Each extra one multiplies testing and support.
- **Never change behavior inside a version**, even when it feels like a fix. Clients may depend on the old behavior. Ship a new version or an opt-in flag.
- **Version client SDKs to match**, so upgrading the SDK is how most clients move to v2.
- **Log the version on every request**, so you always know who is still on the old one.

COMMON MISTAKES

- Bumping the version for additive changes that were never breaking.
- Copy-pasting the whole codebase for v2, so every fix must be made twice.
- “Fixing” behavior inside v1 and breaking clients who relied on it.
- Switching v1 off on the announced date with no usage data and no brownouts.
- Header versioning behind a CDN without `Vary` on that header: v1 and v2 responses get mixed in the cache.

7 Key terms

Breaking change: existing clients fail without code changes

Additive change: new optional field, endpoint or parameter

Tolerant reader: client that ignores unknown fields

Media-type versioning: version inside the Accept header

Date versioning: account pinned to a dated API version

Adapter: thin layer translating old ↔ new format

Deprecation header: says this version is deprecated

Sunset header: date the endpoint will stop working

Brownout: planned short outage of the old version

8 Interview questions

Q1. How do you version an API without breaking clients? BASIC

Prefer additive, backward-compatible changes so you rarely need a version. When you must break, publish v2 side by side with a shared core and a thin v1 adapter, then deprecate v1 with notice (e.g. 12 months), Deprecation and Sunset headers, per-client usage tracking and brownouts.

Q2. Which changes are breaking? BASIC

Removing or renaming a field, changing a type (12.50 dollars → 1250 cents) or a meaning, and making a parameter required. Adding an optional field, endpoint or parameter is safe if clients ignore unknown fields.

Q3. What are the pros and cons of URL versioning? BASIC

Pros: visible in logs, testable in a browser, trivial to route at the gateway, caches like any URL. Cons: versions the whole API at once, forces every client to change URLs, mixes format into resource names, and encourages big infrequent versions.

Q4. You need v2 of one endpoint out of 80. What do you do? SCENARIO

First try an additive change (new field next to the old one). If it must break, header or date versioning lets you version just that change; with URL versioning, serve /v2 for the whole API but route the 79 unchanged endpoints to the same handlers.

Q5. v1 must be switched off in 2 months but 40 clients still call it. Plan? SCENARIO

Pull per-key usage, contact the top callers directly, send Deprecation/Sunset headers, then run brownouts (1 h, 4 h, 24 h) to surface forgotten jobs. Extend for a few big partners only with a written date.

Q6. A teammate fixes a rounding bug in v1 that changes results by a cent. Problem? TRICKY

Yes: clients may reconcile against the old behavior, so a silent change inside a version breaks them. Put the fix in v2 or behind an opt-in flag, and document it.

Q7. You version by header and put a CDN in front. What can go wrong? TRICKY

The CDN caches by URL, so a v1 response may be served to a v2 client. Send Vary: API-Version (or Accept) or include the version in the cache key; URL versioning avoids this entirely.

Next: SD-C-007 · API Pagination: Offset vs Cursor backendroadmap.com